(because it indicates a problem with the mutex or a lack of system resources), it's also the first operation in each member function. Because no data has been modified, this is safe. Unlocking a mutex can't fail, so that's always safe, and the use of `std::lock_guard<>` ensures that the mutex is never left locked.

The call to `data.push()` ❶ may throw an exception if either copying/moving the data value throws an exception or not enough memory can be allocated to extend the underlying data structure. Either way, `std::stack<>` guarantees it will be safe, so that's not a problem either.

In the first overload of `pop()`, the code itself might throw an `empty_stack` exception ❷, but nothing has been modified, so that's safe. The creation of `res` ❸ might throw an exception though for a couple of reasons: the call to `std::make_shared` might throw because it can't allocate memory for the new object and the internal data required for reference counting, or the copy constructor or move constructor of the data item to be returned might throw when copying/moving into the freshly allocated memory. In both cases, the C++ runtime and Standard Library ensure that there are no memory leaks and the new object (if any) is correctly destroyed. Because you *still* haven't modified the underlying stack, you're still OK. The call to `data.pop()` ❹ is guaranteed not to throw, as is the return of the result, so this overload of `pop()` is exception-safe.

The second overload of `pop()` is similar, except this time it's the copy assignment or move assignment operator that can throw ❺ rather than the construction of a new object and a `std::shared_ptr` instance. Again, you don't actually modify the data structure until the call to `data.pop()` ❻, which is still guaranteed not to throw, so this overload is exception-safe too.

Finally, `empty()` doesn't modify any data, so that's exception-safe.